IE4-DSL PREPERATION SHEET

Chapter 1

Synchronous Serial Communication with an Audio Codec

Stisys Group (please ☒)	Team members
01 ☐ 02 ☐ 03 ☒	Minhazul Islam
Team (please ☒)	
A ☐ B ☐ C ☐	
D ☒ E ☐ F ☐	

Prep Task 1

If you use the system clock from the oscillator of the Modsys 2.0 board ($f_{\mathrm{SYSCLK}} = 100\ \mathrm{MHz}$), you obviously cannot exactly set a sampling frequency 48 kHz, because the ratio of the two frequencies is not exactly a power of two.

- What sampling frequency $\hat{f}_{\mathrm{LRC}}$ can you set that is closest to f_{LRC}=48 kHz?
- Calculate the relative error $\epsilon_{\mathrm{rel}} = (f_{\mathrm{LRC}} - \hat{f}_{\mathrm{LRC}})/f_{\mathrm{LRC}}$.
- What ratio $f_{\mathrm{CLK}}/f_{\mathrm{SYSCLK}}$ did you assume?

Set sampling frequency: 48.828 kHz

Relative error: −1.73%

Ratio f_CLK / f_SYSCLK = 8

Prep Task 2

Design a behavioral model in VHDL of the PAR_TO_SER_LR parallel-to-serial converter and verify it.

- First, simulate the overall system as it was provided to you – incomplete yet, but ready for simulation. If you succeed in the simulation without errors, start modeling the parallel-to-series converter.
- Use the file PAR_TO_SER_LR_DUMMY.vdh as starting point and use the the block diagram from the lecture as a guideline for implementation.
- Please mind, the module should have only one shift register for both channels and no parallel registers.
- Write 3 (non-trivial) test cases that will give you confidence that your parallel-to-serial converter will work. One test case
  - has a meaningful title (for your manager who is responsible for the project) and
  - a detailed description of the scenario (for your intern who is writing the testbench).
  - This includes describing what you need to observe in the waveform viewer to know that the test case has passed.
- Simulate at least these 3 test cases and copy per test case a relevant part of your ModelSim simulation (waveforms) into the preparation sheet, based on which you explain why the test passed (if necessary why you think not).
  Please note: Always simulate your module in the overall system. It is duplicate work to write a testbench only for the parallel-to-serial converter, because a good testbench already exists. Better adapt this one appropriately.

Congratulations if you got this far in your preparation. That was quite a bit of work and you had to familiarize yourself with VHDL modeling and tools. Now you can look calmly at the lab session.

VHDL CODE

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



entity PAR_TO_SER_LR is

generic(W : integer := 16);
```

```
port(
        CLK, SRESETN: in std_logic;
        PAR_IN_L, PAR_IN_R: in std_logic_vector(W-1 downto 0);
         -- control signals for shifting bit out
        SHIFT_OUT     : in std_logic;
         -- control signals for load in parallel word
   SAVE_R_LOAD_L, SAVE_L_LOAD_R   : in std_logic;
        SER_OUT: out std_logic
        );
end PAR_TO_SER_LR;


architecture BEHAVIORAL of PAR_TO_SER_LR is


-- single shift reg for both channels
signal SHIFT_REG_NEXT,SHIFT_REG : std_logic_vector( W-1 downto 0);


begin


-- process defining one shift reg
P2S_REGS: process(CLK)
begin
        if CLK = '1' and CLK'event then
                if SRESETN = '0' then
```

```
                        SHIFT_REG <= (others=>'0') after 2 ns;
        else
                        if SHIFT_OUT='1' then
                                SHIFT_REG <= SHIFT_REG_NEXT after 2 ns;
                        end if;
                end if;
            end if;
end process P2S_REGS;


-----------------------------------------------------------------------
-- insert your code in the process below for next-state logic
-- and output logic
-----------------------------------------------------------------------
P2S_COMBIN: process(SHIFT_REG, PAR_IN_L, PAR_IN_R,
                SHIFT_OUT, SAVE_R_LOAD_L, SAVE_L_LOAD_R)
    begin
        -- default: hold current value (no operation)
        SHIFT_REG_NEXT <= SHIFT_REG after 2 ns;


        -- Priority 1: Load left channel word into shift register
        if SAVE_R_LOAD_L = '1' then
            SHIFT_REG_NEXT <= PAR_IN_L after 2 ns;
```

```
    -- Priority 2: Load right channel word into shift register
    elsif SAVE_L_LOAD_R = '1' then
       SHIFT_REG_NEXT <= PAR_IN_R after 2 ns;

    -- Priority 3: Shift MSB out, fill LSB with '0'
    elsif SHIFT_OUT = '1' then
       SHIFT_REG_NEXT <= SHIFT_REG(W-2 downto 0) & '0' after 2 ns;

    -- Priority 4 (default above): hold
    end if;

    -- Output: always MSB of current register (not next)
    SER_OUT <= SHIFT_REG(W-1) after 2 ns;

  end process P2S_COMBIN;


end BEHAVIORAL;
```

TEST CASES

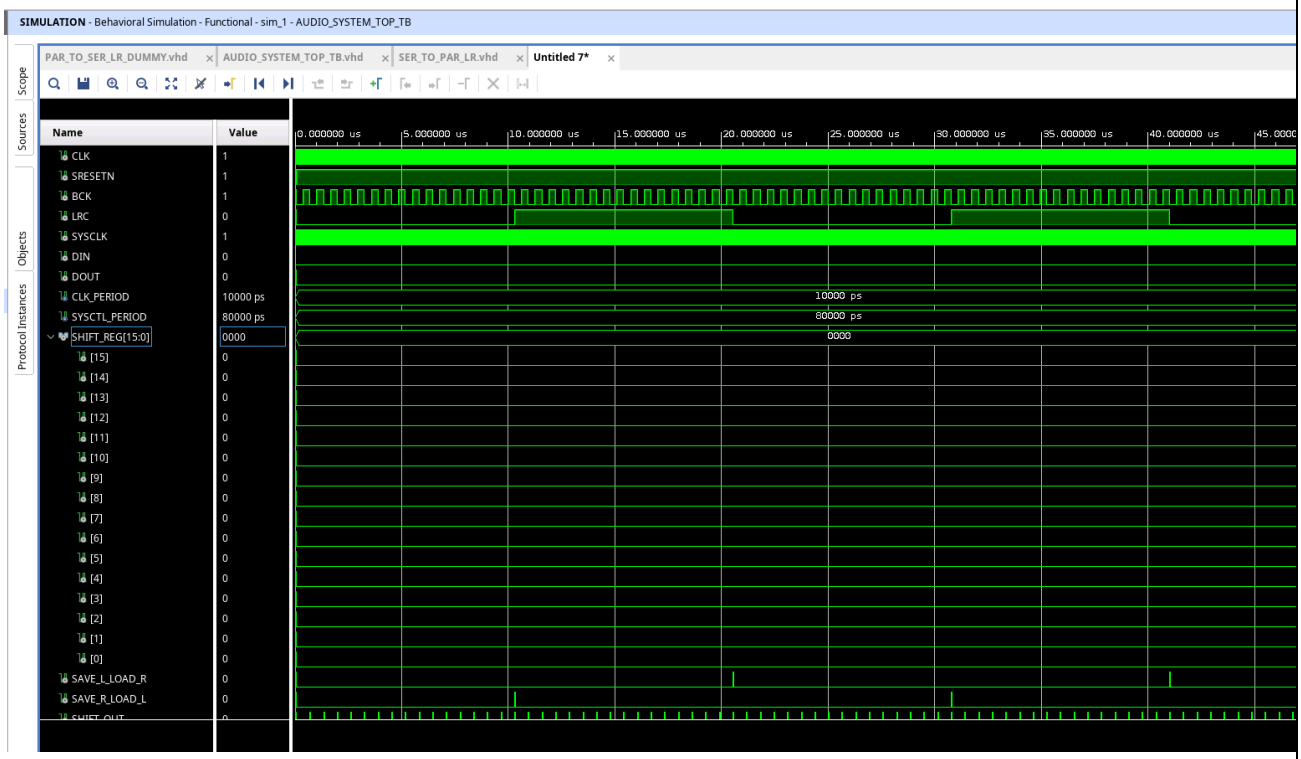
A test case consists of a short, concise title (for the project manager) and a detailed description of how to implement this case (for the HAW intern).

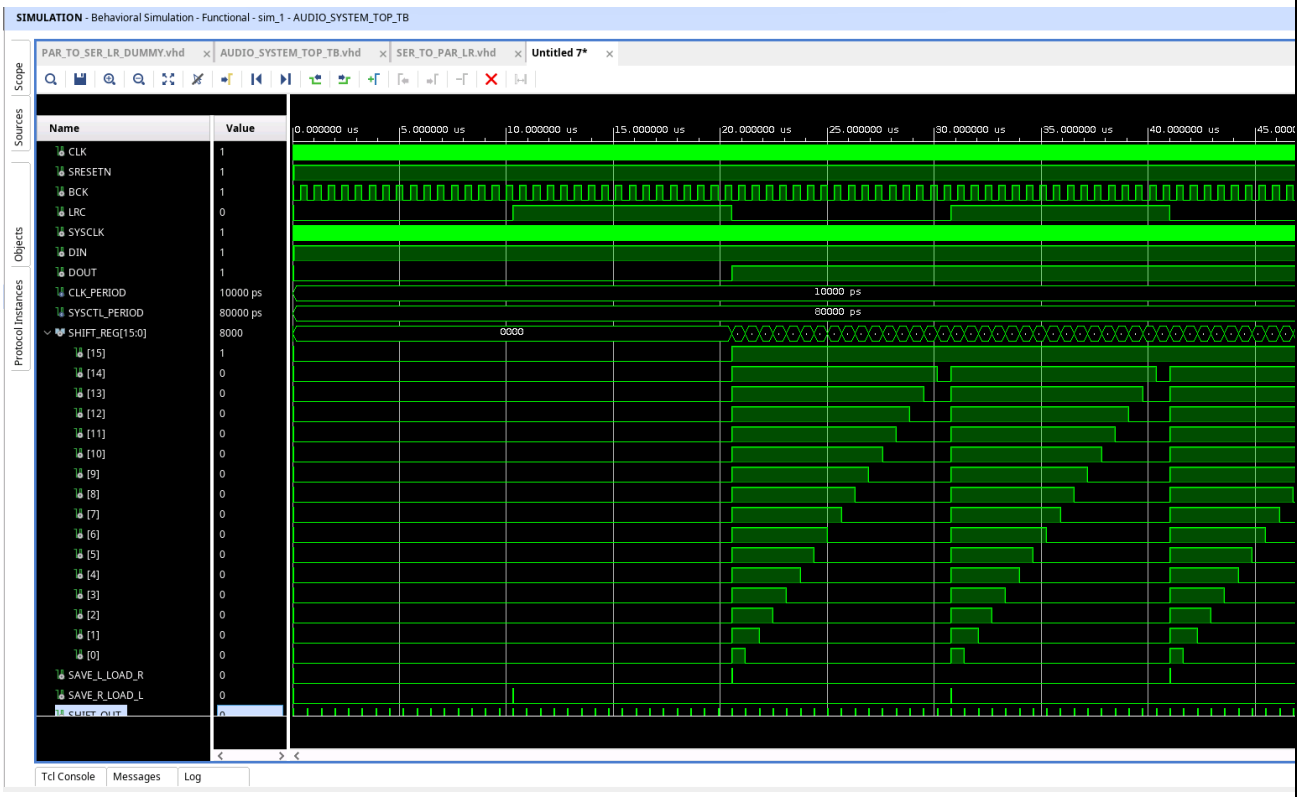
Test case 1	Concise and precise title (max. 10 words): **Title:** All-Zeros Word – Reset and Silence
	Description: Set the DIN stimulus array in the testbench to 32 consecutive '0' bits (both channels = 0x0000). Because AUDIO_PROCESSING_DUMMY echoes PAR_OUT directly back to PAR_IN, the PAR_TO_SER_LR module will be loaded with 0x0000 for both channels. Run the simulation for 50 µs. The test passes if DOUT stays '0' for every bit period throughout both LRC frames, SHIFT_REG reads 0x0000 in hex at all times, SHIFT_OUT pulses exactly 16 times per channel half-frame, and SAVE_R_LOAD_L and SAVE_L_LOAD_R each produce exactly one pulse per LRC period (~20 µs spacing). No '1' may appear on DOUT at any point.
Test case 2	Title: All-Ones Word – Maximum Positive Code
	Description: Set the DIN stimulus array to 32 consecutive '1' bits (both channels = 0xFFFF). AUDIO_PROCESSING_DUMMY echoes 0xFFFF back to PAR_IN for both channels. Run the simulation for 50 µs. Note that DOUT will be '0' during the first LRC period (Frame 1) because the shift register still holds the reset value 0x0000 — the echoed word only becomes available after the first full frame is received by SER_TO_PAR_LR. The test passes if from the second LRC period onwards (~20 µs), DOUT = '1' continuously with no '0' glitch anywhere inside a burst, and SHIFT_REG (hex) shows the countdown sequence FFFF → FFFE → FFFC → ... → 0000 between each pair of SAVE pulses, then reloads back to FFFF at each SAVE event. In the individual bit rows of SHIFT_REG this appears as a staircase — bit [15] stays high longest and bit [0] drops first.
Test case 3	Title: Alternating L/R Channels – Independent Serialisation Verified
	Description: Set the DIN stimulus so that the right channel half-frame carries 0xA5A5 (1010_0101_1010_0101, MSB first) and the left channel half-frame carries 0x5A5A (0101_1010_0101_1010, MSB first). These two patterns are bitwise complements of each other, making any L/R channel swap, MSB/LSB reversal, or off-by-one shift count immediately visible as a wrong pattern on DOUT. Run the simulation for 50 µs. Because of the one-frame echo delay, verify DOUT during the second LRC period (~20 µs onwards). The test passes if after each SAVE_R_LOAD_L pulse the next 16 bits on DOUT are 0101_1010_0101_1010 (0x5A5A, the left channel word, MSB first), and after each SAVE_L_LOAD_R pulse the next 16 bits are 1010_0101_1010_0101 (0xA5A5, the right channel word, MSB first). The two bursts must not bleed into each other — the pattern must switch cleanly at every SAVE boundary. In

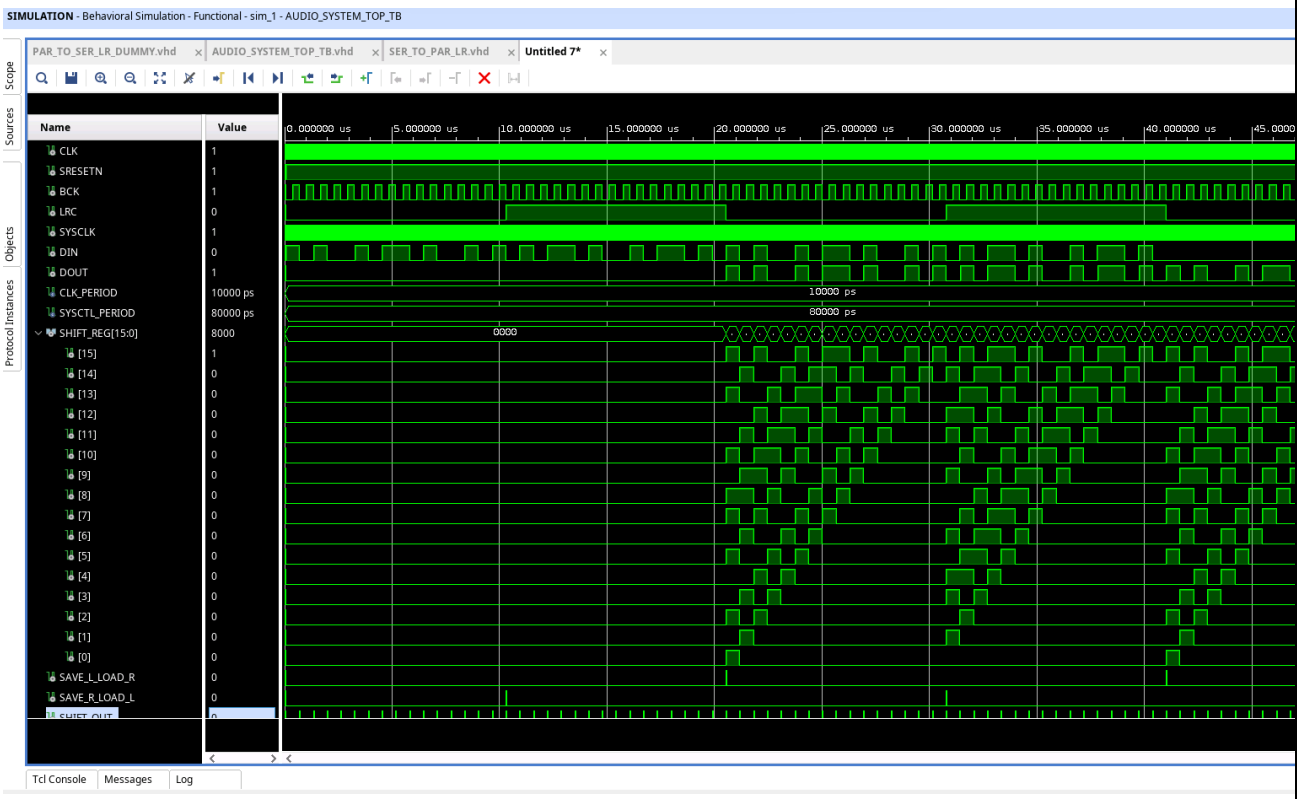
	SHIFT_REG the individual bit rows show a repeating checkerboard pattern, and exactly 16 SHIFT_OUT pulses must appear between each pair of SAVE pulses.

SIMULATION RESULTS

Please mind again to **only simulate the overall system,** i.e. include the parallel-to-serial converter in your overall system and use the provided test bench for simulation. Adapt it to your needs, i.e. to your 3 test cases. Paste in the table below the relevant waveform with which you can prove that your test cases passed successfully.

Test case 1	Waveform (Relevant screenshot of waveforms from ModelSim)  Explanation, why successful. **Test Case 1 — "All-Zeros Word – Reset and Silence"** **Waveform observation:** After reset (SRESETN rises to '1'), DIN is driven to '0' for all 32 bit periods. DOUT remains '0' for the entire 50 µs simulation. SHIFT_REG[15:0] stays at 0x0000 throughout — it is loaded with all-zeros from PAR_IN at every SAVE_R_LOAD_L and SAVE_L_LOAD_R pulse, and shifting an all-zero word left produces all-zeros again. SHIFT_OUT pulses regularly at BCK rate, SAVE_R_LOAD_L and SAVE_L_LOAD_R each pulse exactly once per LRC period (~20 µs apart). **Why the test passed:** The module correctly loaded 0x0000 into the shift register at every channel boundary and serialised all 16 bits as '0' on DOUT. The fact that DOUT never produced a '1' confirms that no spurious data was introduced by the reset state, by the load

	logic, or by the shift operation. The regular SHIFT_OUT pulses confirm that exactly 16 bits were shifted out per channel half-frame before each reload.
Test case 2	Waveform (Relevant screenshot of waveforms from ModelSim)  Explanation, why successful. **Test Case 2 — "All-Ones Word – Maximum Positive Code"** **Waveform observation:** DIN is '1' for all 32 bit periods. During Frame 1 (0–~20 µs) DOUT = 0 because the shift register is still loaded with the reset value 0x0000 — the echoed 0xFFFF from AUDIO_PROCESSING_DUMMY only becomes available after the first full frame has been received by SER_TO_PAR_LR. From Frame 2 (~20 µs onwards) DOUT = 1 continuously with no glitches. SHIFT_REG[15:0] shows the expected countdown: loaded with 0xFFFF at each SAVE pulse, then counting down FFFF → FFFE → FFFC → … → 0000 as bits are shifted out MSB first. This staircase pattern is clearly visible in the individual bit rows — bit [15] stays high longest, bit [0] drops first. **Why the test passed:** The one-frame echo delay is correct hardware behaviour, not an error. Once the echoed word is available, DOUT is '1' for every single bit across both channel bursts, confirming that all 16 bits of 0xFFFF were serialised correctly with no dropped or repeated bits. The staircase countdown in SHIFT_REG confirms that MSB-first serialisation works correctly — each left shift moves the next bit into position [W-1] which drives SER_OUT, and the vacated LSB is filled with '0' as specified.

Test case 3	Waveform (Relevant screenshot of waveforms from ModelSim)  Explanation, why successful. **Test Case 3 — "Alternating Channels – Independent L/R Serialisation"** **Waveform observation:** DIN shows a clearly alternating pattern — the right channel half-frame carries 0xA5A5 = 1010_0101_1010_0101 and the left channel half-frame carries 0x5A5A = 0101_1010_0101_1010. From Frame 2 onwards (~20 µs), DOUT also shows an alternating pattern that switches its phase at every SAVE_R_LOAD_L and SAVE_L_LOAD_R boundary. The individual bit rows of SHIFT_REG show a repeating checkerboard pattern — alternating high and low pulses — which is the visual signature of the 1010… and 0101… patterns being shifted out. The pattern changes cleanly at each SAVE pulse with no bleed between bursts. **Why the test passed:** The complementary patterns 0xA5A5 and 0x5A5A make any fault immediately visible. A swap of the L and R channels would produce the wrong pattern on each burst. A bit reversal (LSB-first instead of MSB-first) would turn 0xA5A5 into 0xA5A5 mirrored, which for this symmetric pattern would still be detectable from the phase of the alternation. An off-by-one shift count would cause the pattern to slip by one position, corrupting the alternating sequence. None of these faults occurred — the DOUT bursts switched cleanly at every SAVE boundary, the alternating pattern in SHIFT_REG is consistent across all LRC periods, and exactly 16 SHIFT_OUT pulses separate each SAVE event. This confirms that PAR_TO_SER_LR

	correctly handles independent left and right channel words using a single shared shift register.